



How to Recover the Full Plaintext of XCB

Peng Wang¹, Shuping Mao^{2(✉)}, Ruozhou Xu³, Jiwu Jing¹, and Yuewu Wang¹

¹ School of Cryptology, University of Chinese Academy of Sciences, Beijing, China
{p-wang, jwjing, wangyuewu}@ucas.ac.cn

² Beijing Electronic Science and Technology Institute, Beijing, China
maoshuping19@mails.ucas.ac.cn

³ State Grid Information and Telecommunication Branch, Beijing, China
xuruozhou21@mails.ucas.ac.cn

Abstract. XCB, a tweakable enciphering mode, is part of IEEE Std. 1619.2 for shared storage media. We show that all versions of XCB are not secure through three plaintext recovery attacks. A key observation is that XCB behaves like an LRW1-type tweakable block cipher for single-block messages, which lacks CCA security. The first attack targets one-block XCB, using three queries to recover the plaintext. The second one requires four queries to recover the plaintext that excludes one block. The last one requires seven queries to recover the *full* plaintext. The first attack applies to any scheme that follows the XCB structure, whereas the latter two attacks work on all versions of XCB, exploiting the separable property of the underlying universal hash function. To address these flaws, we propose the XCB* structure, an improved version of XCB that adds only two XOR operations. We prove that XCB* is STPRP-secure when using AXU hash functions, SPRPs, and a secure random-IV-based stream cipher.

Keywords: XCB · Tweakable enciphering mode · LRW1 · CCA

1 Introduction

The term *tweakable enciphering mode* (TEM) [18], also referred to as *tweakable wide-block cipher mode* [15], is a length-preserving encryption scheme that provides strong tweakable pseudorandom permutation (STPRP) security. TEM is a permutation for a key and a public input called “tweak”. In an ideal TEM, each tweak independently induces a random permutation. The security of a TEM is defined as its indistinguishability from the ideal one, even when an attacker can select tweaks and make encryption and decryption queries.

TEM is well-suited for applications like full disk encryption. By using the data unit index (e.g., sector number) as a tweak, it induces independent pseudo-random permutations to encrypt each data unit. In the last decade, significant

This paper is a follow-up to the work of Bhati et al. [3], which serves as a foundational basis for our current study.

efforts have been made in designing TEMs and proving their security. Notable constructions based on block ciphers include CMC [18], EME [19], EME* [16], XCB [25, 26], HCTR [35], PEP [6], TET [17], HEH [33], HCH [7], HCI [30], MXCB [30], TCT₁ [34], TCT₂ [34], FMix [4], Adiantum [9], and HCTR2 [10], etc. NIST [8] has recently announced plans to develop and standardize a tweakable enciphering mode called the “accordion mode”, highlighting its ability to accommodate varying input lengths. This initiative has sparked several designs including ddd-AES [12], HCTR+ [11], DbHCTR [23], etc.

The design of TEM often utilizes block ciphers and encounters two primary challenges: incorporating the tweak into the permutation and extending the fixed block length of the block cipher to support variable input lengths.

For the first challenge, if we limit the message length to one block, TEM becomes the foundational concept of a tweakable block cipher (TBC) which was proposed by Liskov et al. in their seminal work [24]. This allows us to focus solely on methods for incorporating tweaks. Common TBC constructions based on block ciphers include LRW1, LRW2 [24], CLRW2 [22], and TNT [2], etc. Among these constructions, it is notable that LRW1 is the only one secure against chosen plaintext attacks (CPA) but not against chosen ciphertext attacks (CCA).

For the second challenge, TEMs typically utilize three frameworks to build a variable-input-length permutation [7]: Encrypt-Mix-Encrypt, Hash-CTR-Hash, and Hash-ECB-Hash. Here, “Hash” refers to a *universal hash function*, which is a component more efficient than a block cipher in the construction of modes.

XCB was initially proposed by McGrew et al. [25] in 2004 without a security proof. We refer to it as XCBv1. In 2007, they proposed another version [26], referred to as XCBv2, with a security proof of STPRP. Subsequently, in 2010, XCBv2 was included into the IEEE 1619.2 standard [20].

In 2013, Chakraborty et al. [5] showed that XCBv2 is not secure when the message length is not a multiple of block length, using a distinguishing attack. They also gave security proofs for XCBv1 and the multiple-block-length XCBv2, referred to as XCBv2fb.

Recently, Bhati et al. [3] proposed an attack against XCBv2fb that can recover the plaintext, except for one block data, using only two queries. So it is a *partial* plaintext recovery attack. Their attack is also applicable to HCI [30] and MXCB [30].

In summary, even though the security of XCBv2 [26] and XCBv2fb [5] has been compromised, the current attacks do not pose a threat to XCBv1. Therefore, according to existing literature, XCBv1 [25] is still considered secure. Additionally, there are no *full* plaintext recovery attacks on any version of XCB.

XCB consists of two algorithms: encryption **E** and decryption **D**. Specifically, **E** takes a key K , a tweak T and a plaintext P as input and outputs a ciphertext $C = \mathbf{E}_K^T(P)$. $\mathbf{D}_K^T$ is the inverse of $\mathbf{E}_K^T$. For simplicity, we will omit the key in the notation for both encryption and decryption in the following discussion.

A Warm-Up Full Plaintext Recovery Attack. We first present a *full* plaintext recovery attack on XCBv2bf following the method in [3] by making an addi-

tional decryption query. The attack can retrieve the full plaintext. However, the attack still does not work on XCBv1.

Our main contribution is the proposal of three plaintext recovery attacks applicable to all versions of XCB. These attacks follow a *uniform* pattern of alternate querying $\mathbf{D}^{T'}$ and $\mathbf{E}^T$, with variations in the number of queries and the length of the message.

The key observation is that XCB, when processing a single-block message, behaves similarly to an LRW1-type tweakable block cipher, lacking CCA security.

Attack 1. When the message length is one block, similar to the distinguishing attack on LRW1 described in the paper presentation of [21], we have the equation:

$$\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T = I,$$

where I is an identity transformation. Of course, we can use this equation to distinguish XCB from the corresponding ideal TEM. We reformulate it as $\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} = \mathbf{D}^T$. Hence, given (T, C) , the corresponding plaintext can be retrieved in the following manner: first, use C to query $\mathbf{D}^{T'}$; then, query $\mathbf{E}^T$ using the obtained result; and finally, query $\mathbf{D}^{T'}$ again. This process results in obtaining $\mathbf{D}^T(C)$.

Attack 1 is independent of the specific properties of the components in XCB, and therefore, it can be applied to any scheme that follows the XCB structure, including XCBv1 [25], XCBv2 [26] and XCBv2fb [5].

Attack 2. For a longer ciphertext, such as a typical sector length of 512 bytes, how can we recover the plaintext? By making an additional query to $\mathbf{E}^T$, we can recover the plaintext except for one block.

The attack exploits the separable property of the universal hash function (UHF) in XCB. In simple terms, any string XORed with the input of the UHF h can be separated from the function, as demonstrated by the equation:

$$h(X \oplus \Delta_1, Y \oplus \Delta_2) = h(X, Y) \oplus g(\Delta_1, \Delta_2),$$

where g is a keyed function that is independent of both X and Y .

Attack 3. Surprisingly, by executing three additional queries, we can recover the entire plaintext of XCB. The reason is the following equation,

$$\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} = \mathbf{D}^T.$$

Attack 2 and 3 can be applied to any scheme that utilizes the XCB structure with separable UHFs, including XCBv1 [25], XCBv2 [26], XCBv2fb [5], HCI and MXCB [30].

We summarize all attacks in Table 1. In particular, regarding the standard version of XCB [20], we present the first *full* plaintext attack. Please note that the three attacks are quite *general* in nature; they only require knowledge of the XCB structure (Attack 1) or an additional separable property of UHFs (Attacks 2 and 3), without the necessity for further algorithmic details. In the following, when we mention XCB, it means all versions of XCB.

Table 1. Summary of Attacks on XCB. We note that Attack 1 is applicable to all XCB-type schemes and Attack 2 & 3 are applicable to all XCB-type schemes with separable UHF, where m is the message length and n is the block length of the block cipher.

	Message length	Number of queries	Recovered bits	Target schemes	Ref.
Attack in [5]	$m > n$	2	N/A	XCBv2 [26]	[5]
Attack in [3]	$m > n$	2	$m - n$	XCBv2 [26], XCBv2fb [5],	[3]
A warm-up attack	$m > n$	3	m	HCI [30], MXCB [30]	4.2
Attack 1	$m = n$	3	n	XCBv1 [25], XCBv2 [26], XCBv2fb [5]	5.3
Attack 2	$m > n$	4	$m - n$	XCBv1 [25], XCBv2 [26],	5.4
Attack 3	$m > n$	7	m	XCBv2fb [5], HCI [30], MXCB [30]	5.5

Modification. We modify the XCB structure into XCB* with only extra two XOR operations. We prove that, when h_1 and h_2 are almost-XOR-universal (AXU) hash functions, e and d are SPRPs, and c is a secure random-IV-based stream cipher, the XCB* structure is an STPRP.

In the following, we give notations and definitions in Sect. 2, the XCB and its components in Sect. 3, previous attacks and a warm-up attack in Sect. 4, three plaintext recovery attacks in Sect. 5, modification in Sect. 6, discussions in Sect. 7 and conclusions in Sect. 8.

2 Preliminaries

Notations. Let $\mathcal{X}$ be a set. Let $x \xleftarrow{\$} \mathcal{X}$ denote selecting an element x from the set $\mathcal{X}$ uniformly at random. Let $\text{Perm}(\mathcal{X})$ be a set of all length-preserving permutations on $\mathcal{X}$. If $\mathcal{X} = \{0, 1\}^n$, we denote $\text{Perm}(\mathcal{X})$ as $\text{Perm}(n)$. Let $\pi \xleftarrow{\$} \text{Perm}(\mathcal{X})$ be a random permutation on $\mathcal{X}$. Let $\text{TPerm}(\mathcal{T}, \mathcal{X})$ be a set of all length-preserving tweakable permutations on $\mathcal{X}$ with tweak space of $\mathcal{T}$. Let $\tilde{\pi} \xleftarrow{\$} \text{TPerm}(\mathcal{T}, \mathcal{X})$ be a random tweakable permutation, so that $\tilde{\pi}(T, \cdot)$ is a random permutation for each $T \in \mathcal{T}$. Let $\mathcal{A}$ be an adversary. Let $\mathcal{A}^{f(\cdot)} \Rightarrow b$ represent an algorithm that performs queries on the oracle f and outputs the bit b .

For a binary string X , $|X|$ denotes the length of X in bits. Let $\text{msb}_r(X)$ and $\text{lsb}_r(X)$ be the r leftmost and the r rightmost bits of X respectively. For $X, Y \in \{0, 1\}^n$, $X \oplus Y$ and XY respectively denote addition and multiplication in $GF(2^n)$. Let $\text{bin}_s(i)$ be the s -bit binary representation of i . $X[a, b]$ denotes the substring of X from the a th bit through the b th, and indexing starts at 0.

Block Cipher. A block cipher (or BC for short) $E : \mathcal{K} \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ is a function with key space $\mathcal{K}$ and message space $\{0, 1\}^n$ such that for every key $K \in \mathcal{K}$, $E(K, \cdot)$ is a permutation on $\{0, 1\}^n$. The inverse E is denoted by D such that $D(K, \cdot)$ is the inverse of $E(K, \cdot)$. We write $E(K, P)$ ($D(K, C)$) as $E_K(P)$ ($D_K(C)$) and sometimes omit K for convenience.

Tweakable Enciphering Mode. A tweakable enciphering mode (or TEM for short) $\mathbf{E} : \mathcal{K} \times \mathcal{T} \times \mathcal{X} \rightarrow \mathcal{X}$ is a function with key space $\mathcal{K}$, tweak space $\mathcal{T}$, and message space $\mathcal{X}$ such that for every key $K \in \mathcal{K}$ and every tweak $T \in \mathcal{T}$, $\mathbf{E}(K, T, \cdot)$ is a length-preserving permutation on $\mathcal{X}$. The inverse of $\mathbf{E}$ is denoted by $\mathbf{D}$ such that $\mathbf{D}(K, T, \cdot)$ is the inverse of $\mathbf{E}(K, T, \cdot)$. We write $\mathbf{E}(K, T, P)$ ($\mathbf{D}(K, T, C)$) as $\mathbf{E}_K^T(P)$ ($\mathbf{D}_K^T(C)$) and sometimes omit K for convenience. If $\mathcal{X}$ is $\{0, 1\}^n$, then the TEM is referred to as a tweakable block cipher (TBC). Additionally, if $\mathcal{T}$ is empty, the TEM becomes a block cipher (BC).

Strong Tweakable Pseudorandom Permutation (STPRP). Here, we consider the adversary $\mathcal{A}$ that can query both the encryption and the decryption oracles. Let $\mathbf{E} : \mathcal{K} \times \mathcal{T} \times \mathcal{X} \rightarrow \mathcal{X}$ be a tweakable enciphering mode. Let $\tilde{\pi} \xleftarrow{\$} \text{TPerm}(\mathcal{T}, \mathcal{X})$. The STPRP advantage of $\mathcal{A}$ is defined as:

$$\text{Adv}_{\mathbf{E}}^{\text{stprp}}(\mathcal{A}) = \Pr_{K \xleftarrow{\$} \mathcal{K}} [\mathcal{A}^{\mathbf{E}_K, \mathbf{D}_K} \Rightarrow 1] - \Pr_{\tilde{\pi} \xleftarrow{\$} \text{TPerm}(\mathcal{T}, \mathcal{X})} [\mathcal{A}^{\tilde{\pi}, \tilde{\pi}^{-1}} \Rightarrow 1].$$

If $\mathcal{T} = \emptyset$, we write the notion of STPRP as SPRP.

Universal Hash Function. $h : \mathcal{K} \times \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}^n$ is ϵ -almost-XOR-universal (ϵ -AXU) if for any given two distinct inputs $(X, Y), (X', Y') \in \mathcal{X} \times \mathcal{Y}$ and any output $Z \in \{0, 1\}^n$,

$$\Pr_{K \xleftarrow{\$} \mathcal{K}} [h_K(X, Y) \oplus h_K(X', Y') = Z] \leq \epsilon.$$

When Z is fixed to 0^n , h is called ϵ -almost universal (ϵ -AU).

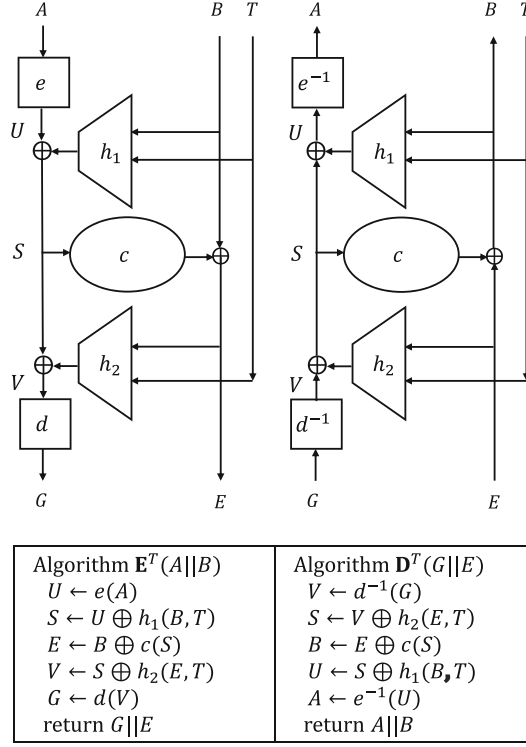
Separable UHF. The universal hash function h is separable if there exists a keyed function g , such that for any X, Y, Δ_1, Δ_2 ($|X| = |\Delta_1|$, $|Y| = |\Delta_2|$), the following equation holds,

$$h_K(X \oplus \Delta_1, Y \oplus \Delta_2) = h_K(X, Y) \oplus g_K(\Delta_1, \Delta_2).$$

3 XCB and Its Components

3.1 The XCB Structure

XCB has two main versions: XCBv1 [25] and XCBv2 [26] (XCBv2fb [5] is the multiple-block-length XCBv2). We provide detailed descriptions of XCBv1 and XCBv2 in Appendix A. To make our attacks as general as possible, we abstract the underlying structure of all versions of XCB, as shown in Fig. 1.

**Fig. 1.** The XCB structure

The encryption $\mathbf{E}$ and decryption $\mathbf{D}$ in the XCB structure use the following components: e , d , c , h_1 and h_2 . e and d are keyed permutations on $\{0, 1\}^n$, and instantiated by the encryption and decryption of a block cipher respectively. Their inverses are e^{-1} and d^{-1} respectively. c is a stream cipher, instantiated by the CTR mode. h_1 and h_2 are two universal hash functions. Note that they all contain keys. For simplicity, we omit the keys in the text and figures. The message of XCB is written as $A||B$, where A is an n -bit string fitting for the block length of the block cipher and B is the rest of the message.

The main differences between the two versions of XCB are as follows. See Appendix A for more details.

- Data sequential arrangement: In XCBv1, following the generic XCB structure, the plaintext is denoted as $A||B$, with its corresponding ciphertext as $G||E$. In contrast, XCBv2 rearranges the plaintext to $B||A$, resulting in the ciphertext being reordered as $E||G$. The sequential arrangement of input and output data does not impact security. For simplicity, we will use the XCBv1 arrangement in the subsequent sections.
- Key size: In XCBv1, the key length is the same as the block length, which means XCBv1 can use AES-128. In contrast, XCBv2's key length matches the key length of the block cipher, allowing it to use all versions of AES.

- Keys for UHF: The keys for the components are generated from the main key. In XCBv1, each of the two UHF employs a different key, while in XCBv2, both functions share the same key.

3.2 Separable UHFs

We first define a common function used by UHFs in XCBv1 and XCBv2.

$$\begin{aligned}
 h_H(X, Y) = & X_1 H^{u+v+1} \oplus X_2 H^{u+v} \oplus \cdots \oplus X_{u-1} H^{v+3} \oplus X_u H^{v+2} \\
 & \oplus Y_1 H^{v+1} \oplus Y_2 H^v \oplus \cdots \oplus Y_{v-1} H^3 \oplus Y_v H^2 \\
 & \oplus (\text{bin}_{\frac{n}{2}}(|X|) \parallel \text{bin}_{\frac{n}{2}}(|Y|))H
 \end{aligned} \tag{1}$$

where $X_1 \parallel X_2 \parallel \cdots \parallel X_u = \text{pad}(X)$, $Y_1 \parallel Y_2 \parallel \cdots \parallel Y_v = \text{pad}(Y)$, $|X_i| = n$, $|Y_j| = n$, $i = 1, 2, \dots, u$ and $j = 1, 2, \dots, v$. If $|X|$ is a multiple of n , then $\text{pad}(X) = X$. Otherwise, $\text{pad}(X) = X \parallel 0^r$, where r is the smallest number required to make $|X \parallel 0^r|$ a multiple of n .

In XCBv1, h_1 and h_2 are defined as:

$$h_i(X, T) = h_{H_i}(X, T), i = 1, 2,$$

where H_1 and H_2 are two distinct keys for UHFs.

In XCBv2, h_1 and h_2 are defined as:

$$\begin{aligned}
 h_1(X, T) &= h_H(0^n \parallel T, \text{pad}(X) \parallel 0^n), \\
 h_2(X, T) &= h_H(T \parallel 0^n, \text{pad}(X) \parallel \text{bin}_{\frac{n}{2}}(|T \parallel 0^n|) \parallel \text{bin}_{\frac{n}{2}}(|X|)),
 \end{aligned}$$

where H is the same key shared by UHFs.

Then we have the following lemma.

Lemma 1. *The universal hash functions h_1, h_2 in XCBv1 and XCBv2 are separable.*

Proof. Note that Theorem 2 in [26] mentions that the universal hash functions in XCBv2 are “linear”, and this property naturally ensures their separability. We first prove the separability of h . From (1) we have

$$\begin{aligned}
 & h(X \oplus \Delta_1, Y \oplus \Delta_2) \\
 = & (X_1 \oplus \Delta_{11}) H^{u+v+1} \oplus \cdots \oplus (X_u \oplus \Delta_{1u}) H^{v+2} \oplus (Y_1 \oplus \Delta_{21}) H^{v+1} \oplus \cdots \\
 & \oplus (Y_v \oplus \Delta_{2v}) H^2 \oplus (\text{bin}_{\frac{n}{2}}(|X \oplus \Delta_1|) \parallel \text{bin}_{\frac{n}{2}}(|Y \oplus \Delta_2|))H \\
 = & X_1 H^{u+v+1} \oplus \cdots \oplus X_u H^{v+2} \oplus Y_1 H^{v+1} \oplus \cdots \oplus Y_v H^2 \\
 & \oplus (\text{bin}_{\frac{n}{2}}(|X|) \parallel \text{bin}_{\frac{n}{2}}(|Y|))H \\
 & \oplus \Delta_{11} H^{u+v+1} \oplus \cdots \oplus \Delta_{1u} H^{v+2} \oplus \Delta_{21} H^{v+1} \oplus \cdots \oplus \Delta_{2v} H^2 \\
 = & h(X, Y) \oplus g(\Delta_1, \Delta_2),
 \end{aligned}$$

where $X_1 \parallel \dots \parallel X_u = \text{pad}(X)$, $\Delta_{11} \parallel \dots \parallel \Delta_{1u} = \text{pad}(\Delta_1)$, $Y_1 \parallel \dots \parallel Y_v = \text{pad}(Y)$, $\Delta_{21} \parallel \dots \parallel \Delta_{2v} = \text{pad}(\Delta_2)$ and

$$g(\Delta_1, \Delta_2) = \Delta_{11}H^{u+v+1} \oplus \cdots \oplus \Delta_{1u}H^{v+2} \oplus \Delta_{21}H^{v+1} \oplus \cdots \oplus \Delta_{2v}H^2.$$

In XCBv1, h_1, h_2 use h directly with two keys, so they are separable. In XCBv2, it is easy to verify that $\text{pad}(X \oplus \Delta) = \text{pad}(X) \oplus \text{pad}(\Delta)$ for any X and Δ satisfying $|X| = |\Delta|$, indicating that they are also separable. $\square$

In the following text, if h_1 and h_2 are separable, we denote

$$\begin{aligned} h_1(X \oplus \Delta_1, T \oplus \Delta_2) &= h_1(X, T) \oplus g_1(\Delta_1, \Delta_2), \\ h_2(X \oplus \Delta_1, T \oplus \Delta_2) &= h_2(X, T) \oplus g_2(\Delta_1, \Delta_2). \end{aligned}$$

We note that all versions of XCB, including XCBv1, XCBv2 and XCBv2fb follow the XCB structure in Fig. 1 and their UHF's are all separable.

4 Previous Attacks on XCB and A New Attack

4.1 Previous Attacks

Recall that, in XCBv2, h_1 and h_2 are defined as:

$$\begin{aligned} h_1(X, T) &= h_H(0^n \| T, \text{pad}(X) \| 0^n), \\ h_2(X, T) &= h_H(T \| 0^n, \text{pad}(X) \| \text{bin}_{\frac{n}{2}}(|T \| 0^n|) \| \text{bin}_{\frac{n}{2}}(|X|)), \end{aligned}$$

where H is the same key shared by UHF's. $|X|$ is a multiple of n , so $\text{pad}(X) = X$. h_1 and h_2 are separable and use the same key, therefore there exist a common function g such that $h_1(X \oplus \Delta, T) = h_1(X, T) \oplus g(\Delta, 0)$ and $h_2(X \oplus \Delta, T) = h_2(X, T) \oplus g(\Delta, 0)$.

Chakraborty et al. [5] observed that the padding function of pad is not injective. For example, for $B = 0^{3n}$ and $B' = 0^{3n-1}$, $\text{pad}(B) = \text{pad}(B') = 0^{3n}$. Therefore, in XCBv2, h_1 is not an almost universal hash function, due to the equation $h_1(B, T) = h_1(B', T)$. If we query the encryption oracle with $(T, A \| B)$ and $(T, A \| B')$, we get $G \| E$ and $G' \| E'$ respectively. The component c receives the same input, resulting in identical outputs. Therefore, $\text{msb}_n(B \oplus E) = \text{msb}_n(c(e(A) \oplus h_1(B, T))) = \text{msb}_n(c(e(A) \oplus h_1(B', T))) = \text{msb}_n(B' \oplus E')$. So, we can use the equation of $\text{msb}_n(B \oplus E) = \text{msb}_n(B' \oplus E')$ to distinguish XCBv2 from a tweakable random permutation using two encryption queries. To prevent the attack, they restricted the message to multiples of the block length and called it XCBv2fb. They also provided security proofs for both XCBv1 and XCBv2fb.

Recently, Bhati et al. [3] gave a *partial* plaintext recovery attack against XCBv2fb with two queries. Given $(T, G \| E)$ (the corresponding plaintext is $A \| B$), the attacker perform the following steps:

- Query the decryption oracle $\mathbf{D}$ with $(T, G \| (E \oplus \Delta))$ and get $A_1 \| B_1$.
- Query the encryption oracle $\mathbf{E}$ with $(T, A_1 \| (B_1 \oplus \Delta))$ and get $G_2 \| E_2$.
- Output $B = E \oplus B_1 \oplus E_2 \oplus \Delta$.

Due to the separable property of UHF's and the fact that h_1 and h_2 share the same key, the input differential to the component c induced during the decryption query, canceled out by the differential induced in the encryption query. Hence, $B \oplus E = (B_1 \oplus \Delta) \oplus E_2$, allowing the recovery of the *partial* plaintext B .

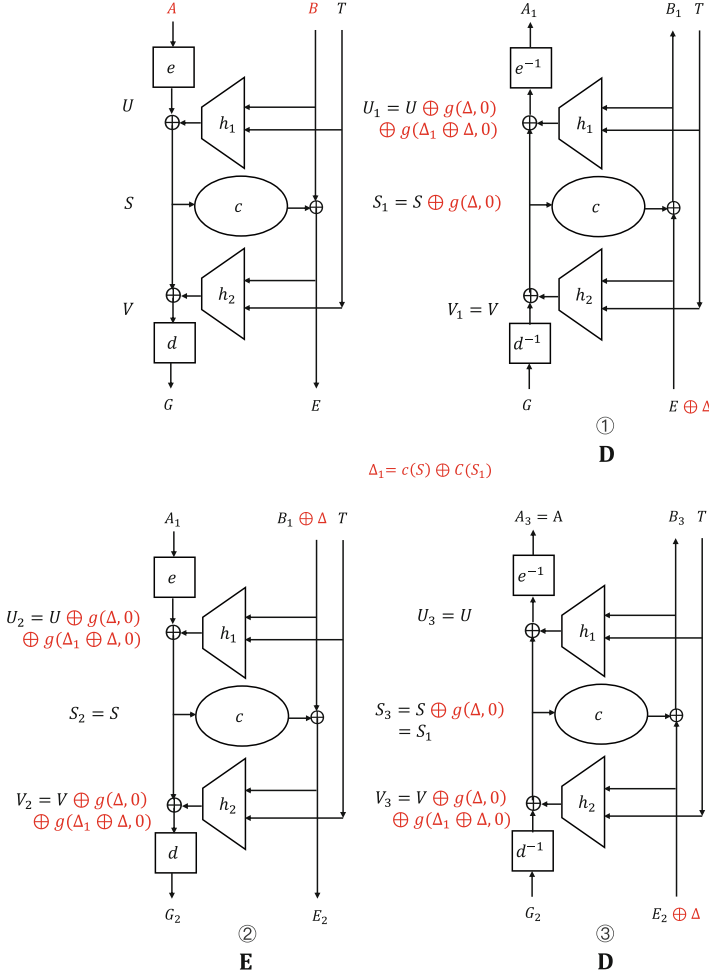


Fig. 2. The process of full plaintext recovery attack against XCBv2fb.

4.2 A Warm-Up Full Plaintext Recovery Attack on XCBv2fb

If the attacker continues to query the decryption oracle with $(T, G_2 || (E_2 \oplus \Delta))$ and obtain $A_3 || B_3$, then $A = A_3$. Therefore we get a *full* plaintext recovery attack against XCBv2fb with three queries.

We demonstrate the correctness of the attacks through Fig. 2. In the process of encrypting or decrypting with XCB, as shown in Fig. 1, we define three intermediate values: U , S , and V . More specifically, for $\mathbf{E}^T(A || B) = G || E$ or $\mathbf{D}^T(G || E) = A || B$, we have

- $U = e(A)$,
- $V = d^{-1}(G)$,
- $S = U \oplus h_1(B, T) = V \oplus h_2(E, T)$.

For the i -th query, the corresponding three values are denoted as U_i , S_i and V_i , $i = 1, 2, 3$.

The intermediate values in each step were illustrated in Fig. 2. For more details, please refer to Appendix B. We see that $A = A_3$ and $B = E \oplus B_1 \oplus E_2 \oplus \Delta$.

The above attacks all target the standard version XCBv2 or its restricted version XCBv2fb. In XCBv1, h_1 and h_2 use different keys, therefore they do not share a common function g under separability property. After two queries, the intermediate value S_3 no longer go back to S , making the attack in [3] and our warm-up attack ineffective.

In the following, we propose a set of more *general* plaintext recovery attacks that can be applied to all versions of XCB. Consequently, none of the XCB versions, including XCBv1, are secure.

5 Recover the (Full) Plaintext of XCB

5.1 XCB is an LRW1-Type TEM

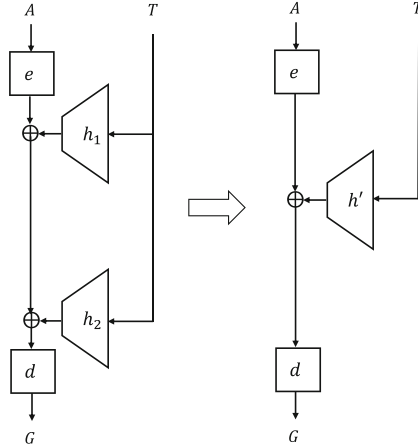


Fig. 3. The encryption $\mathbf{E}$ of XCB when $m = n$.

A key observation is that when we minimize the length of the message to one block (at this point $m = n$ and the input B is an empty string), XCB becomes an LRW1-type construction. Figure 3 gives the encryption of XCB in this case. In this scenario, the hash value of tweak T is XORed into the state between two keyed permutations. Specifically,

$$\mathbf{E}^T(A) = d(e(A) \oplus h'(T)),$$

where $h'(T) = h_1(T) \oplus h_2(T)$.

The encryption of LRW1 is

$$\mathbf{E}^T(P) = e(e(P) \oplus T),$$

where e is a block cipher. LRW1 is a CPA secure tweakable block cipher when e is a PRP, and there exists a 4-query CCA attack to distinguish it from a tweakable random permutation as shown in the paper presentation of [21]¹, in which one slide illustrated that $\mathbf{D}^{T_4}\mathbf{E}^{T_3}\mathbf{D}^{T_2}\mathbf{E}^{T_1} = I$ for tweaks $T_i, i = 1, 2, 3, 4$ satisfying $\bigoplus_{i=1}^4 T_i = 0$. So we can use this property to distinguish LRW1 from a tweakable random permutation.

In XCB ($m = n$), the difference is that the tweak is processed by a UHF. We just choose tweaks satisfying $T_1 = T_3 = T$ and $T_2 = T_4 = T'$, so that

$$\mathbf{D}^{T'}\mathbf{E}^T\mathbf{D}^{T'}\mathbf{E}^T = I$$

for XCB ($m = n$). We can use this property to perform a distinguishing attack with 4 queries. But if we turn the equation into

$$\mathbf{D}^{T'}\mathbf{E}^T\mathbf{D}^{T'} = \mathbf{D}^T,$$

for any tweak-ciphertext pair (T, G) , the queries to $\mathbf{D}^{T'}$, $\mathbf{E}^T$ and $\mathbf{D}^{T'}$ in succession is equivalent to direct decryption of $\mathbf{D}^T$. Therefore, we only use 3 queries to recover the full plaintext. This is Attack 1.

We further extend the attack to the case with arbitrary message lengths, resulting in Attack 2 and Attack 3.

5.2 Overview of Three Attacks

Given a tweak-ciphertext pair $(T, G\|E)$ of XCB, how can one retrieve the corresponding plaintext $A\|B$ without direct decryption? We call it a full recovery plaintext attack when all $A\|B$ can be recovered, and we call it a partial recovery plaintext attack when only A or B can be recovered. In the following, we assume that the message length is m bits and the block length of the block cipher is n bits.

Figure 4 depicts three attacks that share a *uniform* pattern of alternating queries between the decryption and encryption oracles with two different tweaks. Given $(T, G\|E)$, the attacker first queries the decryption oracle with $(T', G\|E)$ and gets $(A_1\|B_1) = \mathbf{D}^{T'}(G\|E)$, where $T' = T \oplus \Delta$ is a different tweak from T , and $\Delta \neq 0$. The attacker then queries the encryption oracle with $(T, A_1\|B_1)$, and gets $(G_2\|E_2) = \mathbf{E}^T(A_1\|B_1)$. So the attacker queries $\mathbf{D}^{T'}$ and $\mathbf{E}^T$ alternatively.

¹ Interestingly, despite assertions in numerous papers that LRW1 lacks CCA security, no concrete attack can be found in the literature. In 2012, during discussions about the design of McOE [13], we presented an attack on LRW1 via email to the authors. As a result, they removed one of McOE instantiations, namely McOE-D, from the paper, as it is based on LWR1. While writing the paper, we only identified an attack in the presentation of [21]. It may have been considered too simple to be worth mentioning.

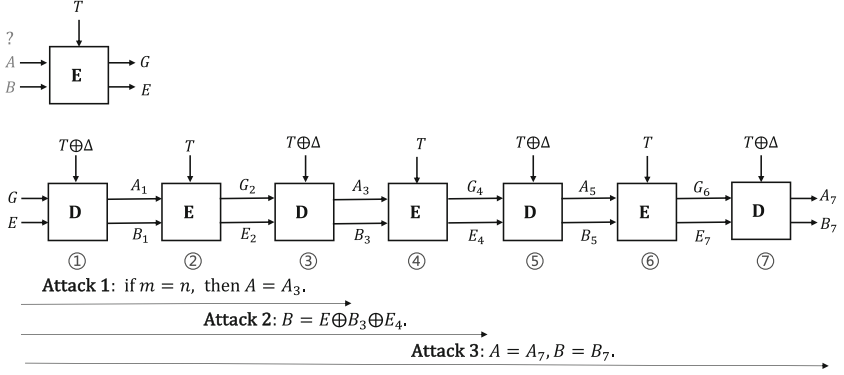


Fig. 4. Three plaintext recovery attacks.

We assume that the attacker gets the following answers: $A_3 \parallel B_3$, $G_4 \parallel E_4$, $A_5 \parallel B_5$, $G_6 \parallel E_6$ and $A_7 \parallel B_7$, as illustrated in Fig. 4.

The attacks differ in the number of queries and the length of the message.

- Attack 1 needs first three queries and the message length $m = n$. The final output A_3 is the full plaintext A .
- Attack 2 needs first four queries and the message length $m > n$. The final output $E \oplus B_3 \oplus E_4$ is the partial plaintext B .
- Attack 3 needs all seven queries and the message length $m > n$. The resulting output, $A_7 \parallel B_7$, is the full plaintext $A \parallel B$.

5.3 Attack 1: Full Plaintext Recovery Attack with 3 Queries ($m = n$)

Attack 1 performs the first three queries with a message length of $m = n$, and the output is A_3 .

The correctness of Attack 1 comes from the following proposition.

Proposition 1. *In XCB, if the message length $m = n$, then for any different T and T' ,*

$$\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} = \mathbf{D}^T.$$

Proof. For $\mathbf{E}^T(A) = G$ or $\mathbf{D}^T(G) = A$, the intermediate values U and V are shown in Fig. 5. The left side of the equation in Proposition 1 corresponds to three queries in Attack 1 and the right side corresponds to the direct decryption. For the i -th query, the intermediate values are U_i and V_i .

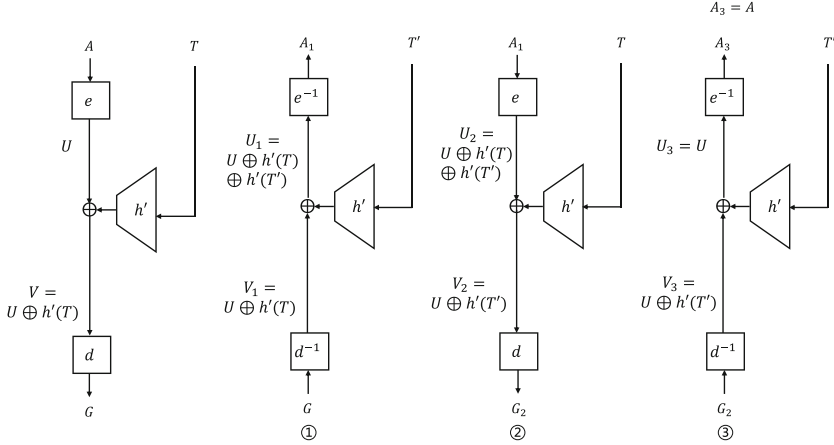


Fig. 5. The process of $\mathbf{D}^{T'}$, $\mathbf{E}^T$, $\mathbf{D}^{T'}$ (Query 1 to Query 3).

For distinct T and T' , the intermediate values before and during the queries are

$$\begin{aligned}
 U &= e(A), \\
 V &= U \oplus h'(T) = V_1, \\
 U_1 &= U \oplus h'(T) \oplus h'(T') = U_2, \\
 V_2 &= U \oplus h'(T') = V_3, \\
 U_3 &= U.
 \end{aligned}$$

So $A_3 = e^{-1}(U) = e^{-1}(e(A)) = A$. That is, $\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'}(G) = A = \mathbf{D}^T(G)$, so we have $\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} = \mathbf{D}^T$. $\square$

In the descriptions of XCBv1 [25] and XCBv2 [26], the message length is within the range $[n, 2^{39}]$, where n is the block length. In the standard version [20] that employs AES as the underlying block cipher, the message length is the message length is defined as m bits, where m ranges from 128 to 2^{32} and is a multiple of 8. While Attack 1 primarily focuses on an extreme case scenario, it is applicable to all versions of XCB. Moreover, Attack 1 does not rely on any specific properties of the underlying components, it applies to any scheme that follows the XCB structure.

To exploit the method to add the tweak, we restrict the message length as a block in other TEMs. We can categorize the methods into four types: LRW1, LRW2, CLRW2 and TNT in Appendix C. We find that HCI and MXCB [30] are also LRW1-type constructions. Since the minimal message length is set to be two blocks, Attack 1 cannot be applied to HCI and MXCB. However, the following attacks are effective.

5.4 Attack 2: Partial Plaintext Recovery Attack with 4 Queries

Attack 2 executes the first four queries without message length restrictions. The output is $E \oplus B_3 \oplus E_4$.

The correctness of Attack 2 comes from the following proposition.

Proposition 2. *In XCB, for any different T and T' , if h_1, h_2 are separable and*

$$\begin{aligned}\mathbf{E}^T(A\|B) &= G\|E, \\ \mathbf{D}^{T'}\mathbf{E}^T\mathbf{D}^{T'}(G\|E) &= A_3\|B_3, \\ \mathbf{E}^T(A_3\|B_3) &= G_4\|E_4.\end{aligned}$$

Then we have

$$B = E \oplus B_3 \oplus E_4.$$

Proof. We define the intermediate values (U, S, V) and (U_i, S_i, V_i) as before. For different values of T and $T' = T \oplus \Delta$, assuming that $\mathbf{E}^T(A\|B) = G\|E$, we compute the intermediate values. These results are illustrated in Fig. 6.

Before queries, we have

$$\begin{aligned}U &= e(A), \\ S &= U \oplus h_1(B, T) = V \oplus h_2(E, T), \\ V &= d^{-1}(G), \\ c(S) &= B \oplus E.\end{aligned}\tag{2}$$

For Query 1:

$$\begin{aligned}V_1 &= V, \\ S_1 &= S \oplus h_2(E, T) \oplus h_2(E, T \oplus \Delta) = S \oplus g_2(0, \Delta), \\ U_1 &= S_1 \oplus h_1(B_1, T \oplus \Delta) = S \oplus g_2(0, \Delta) \oplus h_1(B, T) \oplus g_1(\Delta_1, \Delta) \\ &= U \oplus g_2(0, \Delta) \oplus g_1(\Delta_1, \Delta), \\ c(S_1) &= B_1 \oplus E,\end{aligned}$$

where

$$\Delta_1 = B \oplus B_1 = c(S) \oplus E \oplus c(S_1) \oplus E = c(S) \oplus c(S_1).$$

For Query 2:

$$\begin{aligned}U_2 &= U_1 = U \oplus g_2(0, \Delta) \oplus g_1(\Delta_1, \Delta), \\ S_2 &= S_1 \oplus h_1(B_1, T \oplus \Delta) \oplus h_1(B_1, T) = S \oplus g_2(0, \Delta) \oplus g_1(0, \Delta), \\ V_2 &= S_2 \oplus h_2(E_2, T) = S \oplus g_2(0, \Delta) \oplus g_1(0, \Delta) \oplus h_2(E, T) \oplus g_2(\Delta_2, 0) \\ &= V \oplus g_1(0, \Delta) \oplus g_2(\Delta_2, \Delta), \\ c(S_2) &= B_1 \oplus E_2,\end{aligned}$$

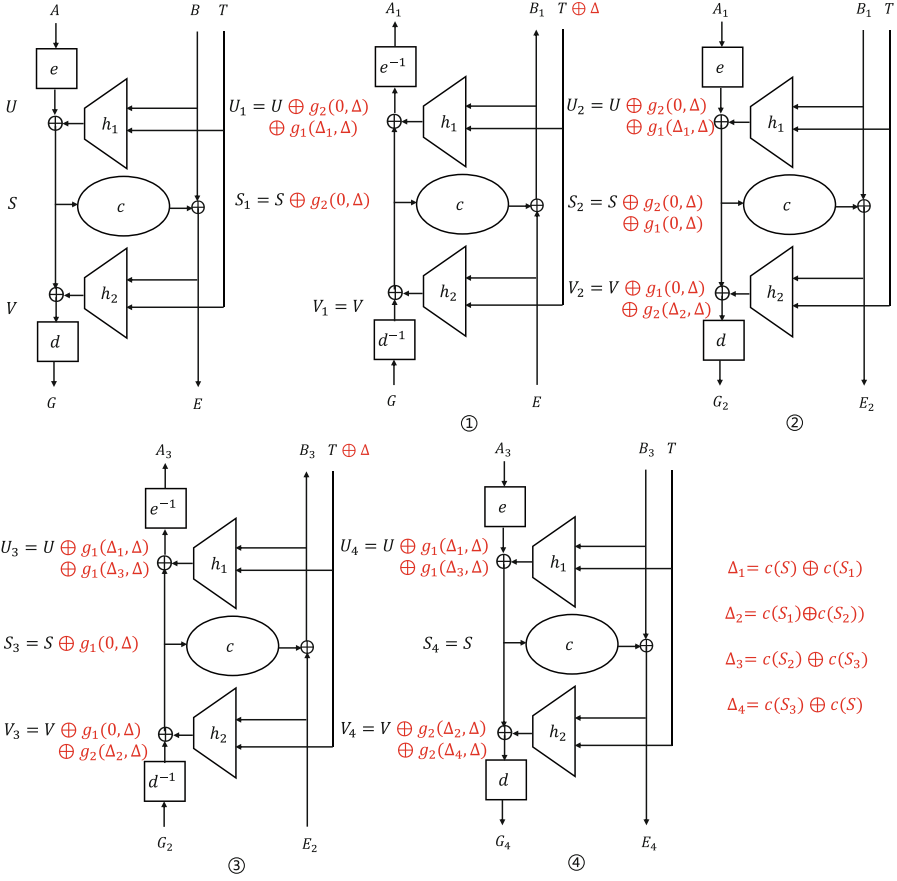


Fig. 6. The process of $D^{T'}$, E^T , $D^{T'}$, E^T (Query 1 to Query 4).

where

$$\Delta_2 = E \oplus E_2 = c(S_1) \oplus B_1 \oplus c(S_2) \oplus B_1 = c(S_1) \oplus c(S_2).$$

For Query 3:

$$\begin{aligned} V_3 &= V_2 = V \oplus g_1(0, \Delta) \oplus g_2(\Delta_2, \Delta), \\ S_3 &= S_2 \oplus h_2(E_2, T) \oplus h_2(E_2, T \oplus \Delta) = S \oplus g_1(0, \Delta), \\ U_3 &= S_3 \oplus h_1(B_3, T \oplus \Delta) = S \oplus g_1(0, \Delta) \oplus h_1(B_1, T) \oplus g_1(\Delta_3, \Delta) \\ &= S \oplus g_1(0, \Delta) \oplus h_1(B, T) \oplus g_1(\Delta_1, 0) \oplus g_1(\Delta_3, \Delta) \\ &= U \oplus g_1(\Delta_1, \Delta) \oplus g_1(\Delta_3, \Delta), \\ c(S_3) &= B_3 \oplus E_2, \end{aligned}$$

where

$$\Delta_3 = B_1 \oplus B_3 = c(S_2) \oplus E_2 \oplus c(S_3) \oplus E_2 = c(S_2) \oplus c(S_3).$$

For Query 4:

$$\begin{aligned} U_4 &= U_3 = U \oplus g_1(\Delta_1, \Delta) \oplus g_1(\Delta_3, \Delta), \\ S_4 &= S_3 \oplus h_1(B_3, T \oplus \Delta) \oplus h_1(B_3, T) = S \oplus g_1(0, \Delta) \oplus g_1(0, \Delta) = S, \\ V_4 &= S_4 \oplus h_2(E_4, T) = S \oplus h_2(E_2, T) \oplus g_2(\Delta_4, 0) \\ &= S \oplus h_2(E, T) \oplus g_2(\Delta_2, 0) \oplus g_2(\Delta_4, 0) = V \oplus g_2(\Delta_2, 0) \oplus g_2(\Delta_4, 0), \\ c(S) &= B_3 \oplus E_4, \end{aligned} \tag{3}$$

where

$$\Delta_4 = E_2 \oplus E_4 = c(S_3) \oplus B_3 \oplus c(S) \oplus B_3 = c(S_3) \oplus c(S).$$

From (2) and (3) we have:

$$c(S) = B \oplus E = B_3 \oplus E_4.$$

So

$$B = E \oplus B_3 \oplus E_4.$$

□

The universal hash functions h_1, h_2 in XCBv1, XCBv2 and XCBv2fb are separable. By Proposition 1, Attack 2 holds for all these versions.

5.5 Attack 3: Full Plaintext Recovery Attack with 7 Queries

In Attack 1, we use 3 queries to recover plaintext A ; in Attack 2, we use 4 queries to recover plaintext B . So the question arises, can we recover the full plaintext $A\|B$?

The answer is yes. Notice that in the 4th query, the intermediate value S_4 goes back to S . Therefore $\Delta_i = c(S_{i-1}) \oplus c(S_i), i = 1, 2, \dots$ will also begin a four-step cycle, where $S_0 = S$. This implies that after an additional four queries, the intermediate values will return to the original (U, S, V) . In fact, in the 7th query we obtain the plaintext of $A\|B$.

For the completeness of the proof, we calculate the intermediate values as before. The correctness of Attack 3 comes from the following proposition:

Proposition 3. *In XCB, for any distinct T and T' , if h_1, h_2 are separable, we have*

$$\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} = \mathbf{D}^T.$$

Proof. This proposition is an extension of Proposition 2, which adds 3 more queries on top of the 4 queries in Proposition 2, proving that querying XCB 7 times can get the entire plaintext directly.

For distinct T and T' , we examine $\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} (G \| E)$. Let $T' = T \oplus \Delta$, the query steps are as follows. Queries 1 to 4 are the same as Proposition 2, and Queries 5 to 7 are shown in Fig. 7.

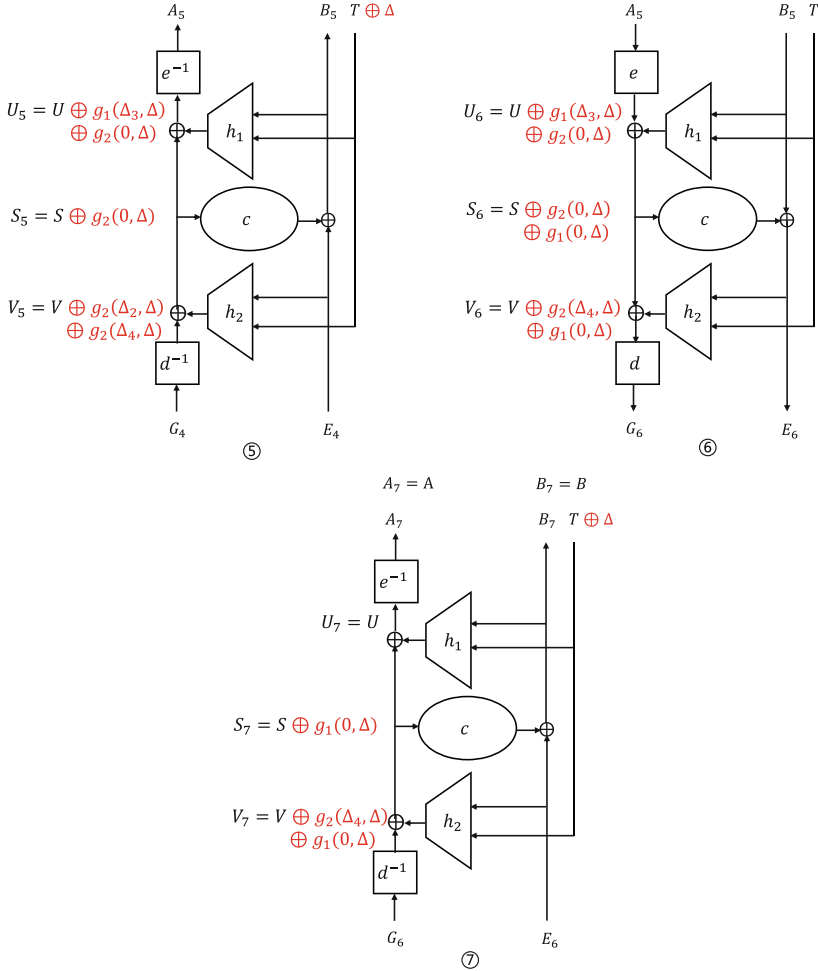


Fig. 7. The process of $\mathbf{D}^{T'}$, $\mathbf{E}^T$, $\mathbf{D}^{T'}$ (Query 5 to Query 7).

For Query 5,

$$\begin{aligned}
V_5 &= V_4 = V \oplus g_2(\Delta_2, 0) \oplus g_2(\Delta_4, 0), \\
S_5 &= S_4 \oplus h_2(E_4, T) \oplus h_2(E_4, T \oplus \Delta) = S \oplus g_2(0, \Delta) = S_1, \\
U_5 &= S_5 \oplus h_1(B_5, T \oplus \Delta) = S_1 \oplus h_1(B_3, T \oplus \Delta) \oplus g_1(\Delta_5, 0) \\
&= S_1 \oplus U_3 \oplus S_3 \oplus g_1(\Delta_1, 0) \\
&= S \oplus g_2(0, \Delta) \oplus U \oplus S \oplus g_1(\Delta_1, 0) \oplus g_1(\Delta_3, \Delta) \oplus g_1(\Delta_1, 0) \\
&= U \oplus g_2(0, \Delta) \oplus g_1(\Delta_3, \Delta), \\
c(S_1) &= B_5 \oplus E_4 = B_1 \oplus E.
\end{aligned}$$

where

$$\Delta_5 = B_3 \oplus B_5 = c(S) \oplus E_4 \oplus c(S_1) \oplus E_4 = c(S) \oplus c(S_1) = \Delta_1.$$

For Query 6,

$$\begin{aligned}
U_6 &= U_5 = U \oplus g_2(0, \Delta) \oplus g_1(\Delta_3, \Delta), \\
S_6 &= S_5 \oplus h_1(B_5, T \oplus \Delta) \oplus h_1(B_5, T) = S \oplus g_2(0, \Delta) \oplus g_1(0, \Delta) = S_2, \\
V_6 &= S_6 \oplus h_2(E_6, T) = S_2 \oplus h_2(E_4, T) \oplus g_2(\Delta_6, 0) \\
&= S_2 \oplus V_4 \oplus S_4 \oplus g_2(\Delta_2, 0) \\
&= S \oplus g_2(0, \Delta) \oplus g_1(0, \Delta) \oplus V \oplus S \oplus g_2(\Delta_2, 0) \oplus g_2(\Delta_4, 0) \oplus g_2(\Delta_2, 0) \\
&= V \oplus g_1(0, \Delta) \oplus g_2(\Delta_4, \Delta), \\
c(S_2) &= B_5 \oplus E_6 = B_1 \oplus E_2,
\end{aligned}$$

where

$$\Delta_6 = E_4 \oplus E_6 = c(S_1) \oplus B_5 \oplus c(S_2) \oplus B_5 = c(S_1) \oplus c(S_2) = \Delta_2.$$

For Query 7,

$$\begin{aligned}
V_7 &= V_6 = V \oplus g_1(0, \Delta) \oplus g_2(\Delta_4, \Delta), \\
S_7 &= S_6 \oplus h_2(E_6, T) \oplus h_2(E_6, T \oplus \Delta) = S \oplus g_1(0, \Delta) = S_3, \\
U_7 &= S_7 \oplus h_1(B_7, T \oplus \Delta) = S_3 \oplus h_1(B_5, T \oplus \Delta) \oplus g_1(\Delta_7, 0) \\
&= S_3 \oplus U_5 \oplus S_5 \oplus g_1(\Delta_3, 0) \\
&= S \oplus g_1(0, \Delta) \oplus U \oplus S \oplus g_1(\Delta_3, \Delta) \oplus g_1(\Delta_3, 0) = U, \\
c(S_3) &= B_7 \oplus E_6 = B_3 \oplus E_2,
\end{aligned}$$

where

$$\Delta_7 = B_5 \oplus B_7 = c(S_2) \oplus E_6 \oplus c(S_3) \oplus E_6 = c(S_2) \oplus c(S_3) = \Delta_3.$$

Then we have

$$A_7 = e^{-1}(U_7) = e^{-1}(U) = e^{-1}(e(A)) = A.$$

and

$$\begin{aligned}
 B_7 &= c(S_3) \oplus E_6 = B_3 \oplus E_2 \oplus E_6 = B_3 \oplus E_2 \oplus B_5 \oplus B_1 \oplus E_2 \\
 &= B_1 \oplus B_3 \oplus B_5 = B_1 \oplus B_3 \oplus E_4 \oplus B_1 \oplus E = E \oplus B_3 \oplus E_4 \\
 &= B.
 \end{aligned}$$

So we have

$$\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} (G \parallel E) = A \parallel B = \mathbf{D}^T (G \parallel E).$$

And

$$\mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} \mathbf{E}^T \mathbf{D}^{T'} = \mathbf{D}^T.$$

□

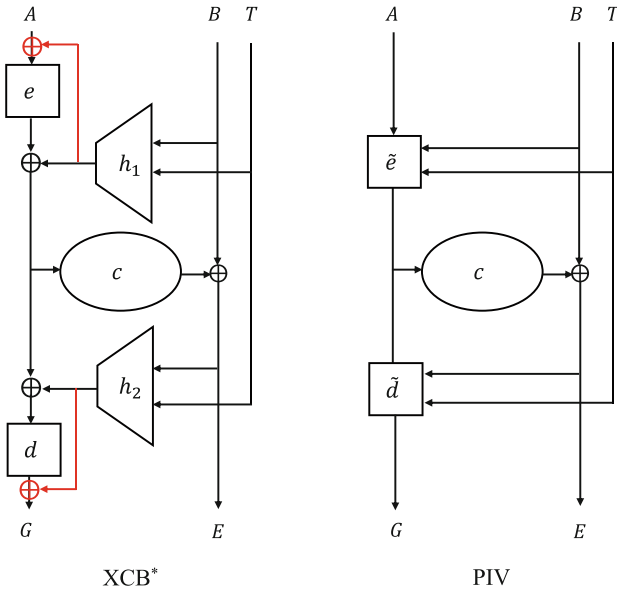


Fig. 8. XCB* and PIV.

6 Modification

All versions of XCB have security vulnerabilities. How can these be addressed? If we want to handle one-block-length messages, we should not use the XCB structure. Attack 1 shows the weakness of the structure. Even if we use stronger components, such as making h_1 and h_2 pseudorandom functions, XCB still fails to achieve CCA security.

Our suggestion is to XOR the output of $h_i, i = 1, 2$ into both before and after the block cipher as shown on the left of Fig. 8. The modification only adds two XOR operations. We call the new structure XCB*. If we look at the UHF and its left block cipher as a whole, it is exactly an LRW2 construction. The same method was used in [1] to strengthen the security of GCM. Consequently, the whole structure changes to the PIV structure proposed by Shrimpton et al. [34]. We prove that when h_1 and h_2 are AXU hash functions, e and d are SPRPs, and c is a secure IV-based stream cipher, then XCB* is an STPRP.

The following is the pseudocode for XCB* and PIV. We note that the components e, d, h_1, h_2, c in XCB* and $\tilde{e}, \tilde{d}, h_1, h_2, c$ PIV, are all independent respectively.

Algorithm 1. The XCB* encryption and decryption

$\text{XCB}^*.\mathbf{E}^T(A\ B)$	$\text{XCB}^*.\mathbf{D}^T(G\ E)$
$\Delta_1 \leftarrow h_1(B, T)$	$\Delta_2 \leftarrow h_2(E, T)$
$S \leftarrow e(A \oplus \Delta_1) \oplus \Delta_1$	$S \leftarrow d^{-1}(G \oplus \Delta_2) \oplus \Delta_2$
$E \leftarrow B \oplus c(S)$	$B \leftarrow E \oplus c(S)$
$\Delta_2 \leftarrow h_2(E, T)$	$\Delta_1 \leftarrow h_1(B, T)$
$G \leftarrow d(V \oplus \Delta_2) \oplus \Delta_2$	$A \leftarrow e^{-1}(S \oplus \Delta_1) \oplus \Delta_1$
return $G\ E$	return $A\ B$

Algorithm 2. The PIV encryption and decryption

$\text{PIV}.\mathbf{E}^T(A\ B)$	$\text{PIV}.\mathbf{D}^T(G\ E)$
$S \leftarrow \tilde{e}^{(B, T)}(A)$	$S \leftarrow \tilde{d}^{-1(E, T)}(G)$
$E \leftarrow B \oplus c(S)$	$B \leftarrow E \oplus c(S)$
$G \leftarrow \tilde{d}^{(E, T)}(S)$	$A \leftarrow \tilde{e}^{-1(B, T)}(B)$
return $G\ E$	return $A\ B$

LRW2 is an STPRP if the underlying block cipher is an SPRP and the UHFs are AXU, as proved by the Theorem 2 in [24].

Lemma 2 (Theorem 2 in [24]). *Let $\tilde{e}[e, h]^T(P) = e(P \oplus h(T)) \oplus h(T)$, where e is a block cipher, and h is ϵ -AXU. Then $\tilde{e}$ is an STPRP. Specifically, for any adversary $\mathcal{A}$ making q queries, there exists an adversary $\mathcal{B}$*

$$\mathbf{Adv}_{\tilde{e}[e, h]}^{\text{stprp}}(\mathcal{A}) \leq \mathbf{Adv}_e^{\text{sprp}}(\mathcal{B}) + 3\epsilon q^2.$$

Lemma 3. *For $\text{PIV}[\tilde{e}, \tilde{d}, c]$, let $\mathcal{A}$ be an adversary making q queries. Then there exist adversaries $\mathcal{B}$, $\mathcal{C}$ and $\mathcal{D}$, each making q queries, such that*

$$\mathbf{Adv}_{\text{PIV}[\tilde{e}, \tilde{d}, c]}^{\text{stprp}}(\mathcal{A}) \leq \mathbf{Adv}_{\tilde{e}}^{\text{stprp}}(\mathcal{B}) + \mathbf{Adv}_{\tilde{d}}^{\text{stprp}}(\mathcal{C}) + \mathbf{Adv}_c^{\text{ivrnd}}(\mathcal{D}) + \frac{3q^2}{2^{n+1}}.$$

Lemma 3 is similar to Theorem 1 in [34]. The difference is the security notion of the stream cipher. In Lemma 3 $S||c(S)$ is indistinguishable from a random string for a random IV S , whereas in Theorem 1 in [34], $c(S)$ is indistinguishable from a random string for a nonce S which never repeats. The security notions based on random IV and nonce are referred to as ivRND and nRND, respectively. They are similar to IV-based encryption (ivE) and nonce-based encryption (nE) in [29], or weak pseudorandom function (wPRF) and pseudorandom function (PRF) in [28, 32]. We choose the weaker security notion due to the fact that the stream cipher in XCBv1 and XCBv2 is ivRND secure but not nRND secure. For the sake of completeness, we provide the proof in Appendix D.

In $\text{PIV}[\tilde{e}, \tilde{d}, c]$, let $\tilde{e}^T(P) = e(P \oplus h_1(T)) \oplus h_1(T)$ and $\tilde{d}^T(P) = d(P \oplus h_2(T)) \oplus h_2(T)$, then $\text{PIV}[\tilde{e}, \tilde{d}, c]$ becomes $\text{XCB}^*[e, h_1, d, h_2, c]$ as shown in Fig. 8. Combine Lemma 2 and 3, we have the following theorem.

Theorem 1. *Assume that h_1 and h_2 are both ϵ -AXU. For $\text{XCB}^*[e, h_1, d, h_2, c]$, let $\mathcal{A}$ be an adversary making q queries. Then there exist adversaries $\mathcal{B}$, $\mathcal{C}$ and $\mathcal{D}$, making q queries, respectively, such that*

$$\text{Adv}_{\text{XCB}^*[e, d, h_1, h_2, c]}^{\text{stprp}}(\mathcal{A}) \leq \text{Adv}_e^{\text{stprp}}(\mathcal{B}) + \text{Adv}_d^{\text{stprp}}(\mathcal{C}) + \text{Adv}_c^{\text{ivrnd}}(\mathcal{D}) + \frac{3q^2}{2^{n+1}} + 6\epsilon q^2.$$

Note that the fixing method works for XCBv1 but is not applicable for XCBv2 due to its UHF h_1 is not an AXU hash function. For the add-tweak method, if we restrict the message length to one block, XCB^* becomes a tweakable block cipher with the Chained LRW2 (CLRW2) construction, which is CCA secure beyond birthday bound [22].

7 Discussions

Requirement of CCA Security. STPRP is an essential security requirement for TEM. As IEEE 1619.2 [20] states the security goal as “Interacting with the encryption and decryption routines, it should be infeasible to distinguish them from a random permutation and its inverse. Moreover, varying the plaintext length and/or the associated data should look like using a different and independent key.”

In NIST’s draft for the requirements for Accordion mode [8], it also sets the security goal as STPRP. The document describes three categories of applications for an accordion mode: authenticated encryption with associated data (AEAD), tweakable encryption for storage devices, and deterministic authenticated encryption. All applications require the underlying TEM to be STPRP-secure.

In full disk encryption, the plaintext P in the sector indexed by T is encrypted into $C = \mathbf{E}^T(P)$. Given (T, C) , assume we can access the encryption oracle $\mathbf{E}^T$ and the decryption oracle of another sector $\mathbf{D}^{T'}$. Then, with four queries, Attack 2 can recover the plaintext that excludes one block. With seven queries, Attack 3 can recover the full plaintext.

STPRP implies security against plaintext recovery attacks. Our findings reveal that XCB does not meet the STPRP criteria, indicating previous security proofs for XCBv2 [26], XCBv2fb and XCBv1 [5] are all flawed.

Flaws in Previous Proofs. The security of STPRP is derived from the indistinguishability between the results of encryption or decryption queries and random strings, as discussed in a series of papers [5, 16, 18, 19]. We assume that the target plaintext is connected by the ciphertext through the 0-th query, and (U_i, S_i, V_i) are the intermediate values for i -th query as shown in Fig. 6 and 7, where $i = 0, 1, 2, \dots$, and $(U_0, S_0, V_0) = (U, S, V)$.

In XCB, if the underlying block cipher is a random permutation, the output randomness for each query depends on the following multi sets: $\mathcal{U}$ consisting of U_i obtained from decryption queries, $\mathcal{V}$ consisting of V_i obtained from encryption queries and $\mathcal{S}$ consisting the inputs to block cipher in the component c , which are determined by S_i obtained from encryption or decryption queries.

The multi sets of $\mathcal{U}$, $\mathcal{V}$ and $\mathcal{S}$ correspond to $\mathcal{R}_1$ (Rng_1), $\mathcal{R}_3$ (Rng_3) and $\mathcal{D}_2$ (Dom_2) respectively in the security proof of XCBv2fb (XCBv2fb) in [5].

The negligible collision probability in $\mathcal{U}$ ($\mathcal{V}$) induces the left-output randomness of each decryption (encryption) query, and $\mathcal{S}$ the right-output randomness. The event Ω defined in the proof of XCBv2 in [26] corresponds to the absence of collisions in $\mathcal{U}$, $\mathcal{V}$ and $\mathcal{S}$.

For Attack 1, in the third query, $U_3 = U$. For Attacks 2 and 3, in the forth query, $S_4 = 4$ and in the seventh query, $U_7 = U$.

As a result, all collision probabilities go to 1, thereby contradicting the proofs in [26] and [5].

Structure Weakness of XCB. The underlying issue with XCB's security lies primarily in its method of integrating tweaks. Though XCB and HCTR are both categorized within the Hash-CTR-Hash framework—leveraging UHF's in the first and third layers, with the CTR mode in the middle—their tweak handling is fundamentally different. XCB adopts the LRW1-type method, while HCTR employs the more secure LRW2-type method. See Appendix C for more detail. The LRW1 construction inherently makes XCB vulnerable to CCA attacks, as demonstrated by Attack 1.

Another critical vulnerability is the separability of UHF's, which allows for attacks on schemes supporting arbitrary message lengths. While one might hope that selecting an appropriate UHF could mitigate these attacks, this is unlikely. Separability is a nearly unavoidable characteristic, as most practical UHF's rely on basic algebraic operations. For instance, GHASH in GCM [27], POLYVAL in AES-GCM-SIV [14], Poly1305 in ChaCha20-Poly1305 [31], and BPE in TET [17] etc. are all separable for some operation.

8 Conclusions

In this paper, we present three plaintext recovery attacks that are applicable to all versions of XCB. When $m = n$, three queries are sufficient to recover the

plaintext A of (T, G) . The attacks are applicable to any scheme utilizing the XCB structure. When $m > n$, four queries are enough to recover the *partial* plaintext B of $(T, G||E)$, and seven queries are enough to recover the *full* plaintext $A||B$ of $(T, G||E)$. These attacks apply to any scheme that uses the XCB structure with separable UHFs.

Hence, in the case of XCBv1, we need seven queries to recover the full plaintext. In contrast, for XCBv2 (XCBv2fb), our warm-up method uses only three queries. We leave it as an open problem to determine whether there are full plaintext recovery attacks against any version of XCB that require fewer queries.

As a final recommendation, we suggest reconsidering the continued use of XCB. Its inherent *structural weaknesses*, particularly in how it manages tweaks and the separability of its UHFs, make it unsuitable for secure applications. A more robust alternative would be to modify the XCB structure by incorporating two additional XOR operations, effectively transforming it into the PIV construction [34]. This adjustment provides a pathway to stronger security guarantees.

Acknowledgments. The authors would like to thank Yaobin Shen for the insightful discussions, which have significantly contributed to the improvement of this paper. We would like to thank all the anonymous reviewers of Eurocrypt 2025 and Crypto 2025 for their helpful suggestions and comments to the paper. The work is supported by the National Key Research and Development Program of China (No. 2023YFB3105802).

A The Description of XCBv1 and XCBv2

The encryption of XCBv1 is described in Algorithm 3, the encryption of XCBv2 is described in Algorithm 4.

Algorithm 3. The XCBv1 encryption operation. Given a key $K \in \{0, 1\}^n$, a plaintext $P \in \{0, 1\}^m$, where $m \in [n, 2^{39}]$, and a tweak $T \in \{0, 1\}^t$, where $t \in [0, 2^{39}]$, this operation returns a ciphertext $C \in \{0, 1\}^m$.

```

 $H_1 \leftarrow e_K(0^{n-3}||001)$ 
 $H_2 \leftarrow e_K(0^{n-3}||011)$ 
 $K_e \leftarrow e_K(0^n)$ 
 $K_d \leftarrow e_K(0^{n-3}||100)$ 
 $K_c \leftarrow e_K(0^{n-3}||010)$ 
 $A \leftarrow P[0, n-1]$ 
 $B \leftarrow P[n, m-1]$ 
 $U \leftarrow e_{K_e}(A)$ 
 $S \leftarrow U \oplus h_{1H_1}(B, T)$ 
 $E \leftarrow B \oplus c_{K_c}(S)$ 
 $V \leftarrow S \oplus h_{2H_2}(E, T)$ 
 $G \leftarrow d_{K_d}(V)$ 
 $C \leftarrow G||E$ 
return  $C$ 

```

Algorithm 4. The XCBv2 encryption operation. Given a key $K \in \{0,1\}^k$, a plaintext $P \in \{0,1\}^m$, where $m \in [n, 2^{39}]$, and a tweak $T \in \{0,1\}^t$, where $t \in [0, 2^{39}]$, this operation returns a ciphertext $C \in \{0,1\}^m$.

```

 $H \leftarrow e_K(0^n)$ 
 $K_e \leftarrow \text{msb}_k(e_K(0^{n-3}||001)||e_K(0^{n-3}||010))$ 
 $K_d \leftarrow \text{msb}_k(e_K(0^{n-3}||011)||e_K(0^{n-3}||100))$ 
 $K_c \leftarrow \text{msb}_k(e_K(0^{n-3}||101)||e_K(0^{n-3}||110))$ 
 $A \leftarrow P[m-n, m-1]$ 
 $B \leftarrow P[0, m-n-1]$ 
 $U \leftarrow e_{K_e}(A)$ 
 $S \leftarrow U \oplus h_{1H}(B, T)$ 
 $E \leftarrow U \oplus c_{K_c}(S)$ 
 $V \leftarrow S \oplus h_{2H}(E, T)$ 
 $G \leftarrow d_{K_d}(V)$ 
 $C \leftarrow E||G$ 
return  $C$ 

```

B Details of the attack in Fig. 2

Before queries, we have

$$\begin{aligned}
 U &= e(A), \\
 S &= U \oplus h_1(B, T) = V \oplus h_2(E, T), \\
 V &= d^{-1}(G), \\
 c(S) &= B \oplus E.
 \end{aligned} \tag{4}$$

For Query 1:

$$\begin{aligned}
 V_1 &= V, \\
 S_1 &= S \oplus h_2(E, T) \oplus h_2(E \oplus \Delta, T) = S \oplus g(\Delta, 0), \\
 U_1 &= S_1 \oplus h_1(B_1, T) = S \oplus g(\Delta, 0) \oplus h_1(B, T) \oplus g(\Delta_1 \oplus \Delta, 0) \\
 &= U \oplus g(\Delta, 0) \oplus g(\Delta_1 \oplus \Delta, 0), \\
 c(S_1) &= B_1 \oplus E \oplus \Delta,
 \end{aligned}$$

where

$$\Delta_1 = B \oplus B_1 \oplus \Delta = E \oplus c(S) \oplus E \oplus \Delta \oplus c(S_1) \oplus \Delta = c(S) \oplus c(S_1).$$

For Query 2:

$$\begin{aligned}
 U_2 &= U_1 = U \oplus g(\Delta, 0) \oplus g(\Delta_1 \oplus \Delta, 0), \\
 S_2 &= U_2 \oplus h_1(B_1 \oplus \Delta, T) = U \oplus g(\Delta, 0) \oplus g(\Delta_1 \oplus \Delta, 0) \oplus h_1(B \oplus \Delta_1, T) \\
 &= U \oplus g(\Delta, 0) \oplus g(\Delta_1 \oplus \Delta, 0) \oplus h_1(B, T) \oplus g(\Delta_1, 0) = U \oplus h_1(B, T) = S, \\
 V_2 &= S_2 \oplus h_2(E_2, T) = S \oplus h_2(E \oplus \Delta_1, T) = V \oplus g(\Delta_1, 0) \\
 &= V \oplus g(\Delta, 0) \oplus g(\Delta_1 \oplus \Delta, 0), \\
 c(S_2) &= c(S) = B_1 \oplus \Delta \oplus E_2.
 \end{aligned}$$

Then from (4), $c(S) = B \oplus E = B_1 \oplus \Delta \oplus E_2$ and $B = E \oplus B_1 \oplus E_2 \oplus \Delta$. This is the partial plaintext recovery attack performed by Bhati et al. [3] on XCBv2fb through two queries. If we perform another decryption query, we can recover all plaintext.

For Query 3:

$$\begin{aligned} V_3 &= V_2 = V \oplus g(\Delta, 0) \oplus g(\Delta_1 \oplus \Delta, 0), \\ S_3 &= S_2 \oplus h_2(E_2, T) \oplus h_2(E_2 \oplus \Delta, T) = S \oplus g(\Delta, 0) = S_1, \\ U_3 &= S_3 \oplus h_1(B_3, T) = S \oplus g(\Delta, 0) \oplus h_1(B \oplus \Delta, T) \\ &= S \oplus h_1(B, T) = U. \end{aligned}$$

Then we have

$$\begin{aligned} B_3 &= E_2 \oplus \Delta \oplus c(S_1) = B_1 \oplus \Delta \oplus c(S) \oplus \Delta \oplus c(S_1) = B_1 \oplus \Delta_1 \\ &= B \oplus \Delta, \\ A_3 &= e^{-1}(U_3) = e^{-1}(U) = e^{-1}(e(A)) = A. \end{aligned}$$

So $B = B_3 \oplus \Delta$ and $A = A_3$.

C Methods to Add Tweaks

In Sect. 5.1, we see that XCB becomes an LRW1-type tweakable block cipher in the case of $m = n$ (Fig. 3), leading to a CCA attack.

By limiting the length of the message to one block, we transform TEM into a tweakable block cipher. This alteration enables us to concentrate exclusively on strategies for incorporating tweaks.

For another example, if we do it in HCTR [35] as in Fig. 9, we can see that HCTR follows the LRW2-type method to add tweak.

We investigate TEM designs based on block ciphers in the current literature and find that there are mainly four methods to add tweaks, including LRW1-type, LRW2-type, CLRW2-type and TNT type. As shown in Fig. 10, the LRW1-type method XORs the tweak between two block ciphers; the LRW2-type method XORs the UHF value of tweak before and after a block cipher; the CLRW2-type method is a chain of two LRW2; the TNT-type method XORs the tweak between two block ciphers twice.

We summarize block-cipher-based TEMs in the following list.

- LRW1-type: XCB [25, 26], HCI [30], MXCB [30];
- LRW2-type: HCTR [35], HCTR2 [10], CMC [18], PEP [6], TET [17], HEH [33], HCH [7], Adiantum [9], DbHCTR [23];
- CLRW2-type: XCB*, TCT₁ [34];
- TNT-type: EME [19], EME* [16].

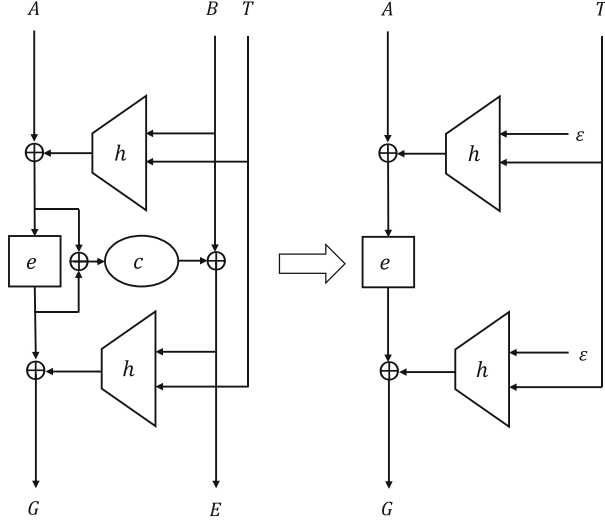


Fig. 9. HCTR and LRW2.

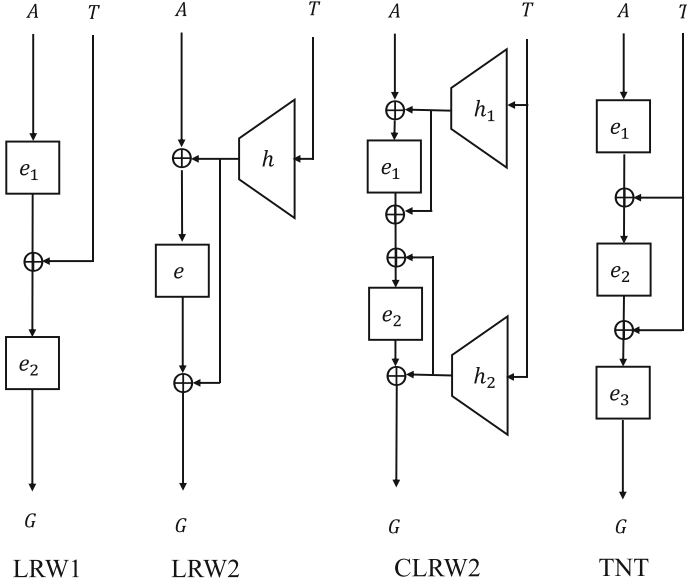


Fig. 10. LRW1, LRW2, CLRW2 and TNT.

D Proof of Lemma 3

ivRND and nRND. Stream cipher c is a keyed variable-output-length function. We define two security notions for stream ciphers, one is ivRND and the other is nRND. The ivRND security is defined through two games ivReal and ivRandom.

When adversary $\mathcal{A}$ queries l , the game ivReal returns $S \parallel \text{msb}_l(c(S))$, where S is an n -bit random string, the game ivRandom returns an $(n + l)$ -bit random string R . The ivRand advantage of $\mathcal{A}$ is defined as $\mathbf{Adv}_c^{\text{ivrnd}}(\mathcal{A}) = \Pr[\mathcal{A}^{\text{ivReal}} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{ivRandom}} \Rightarrow 1]$.

The nRND security is defined through two games nReal and nRandom . When adversary $\mathcal{A}$ queries (S, l) , the game nReal returns $\text{msb}_l(c(S))$, the game nRandom returns an l -bit random string R . The adversary $\mathcal{A}$ never repeats S . The nRand advantage of $\mathcal{A}$ is defined as $\mathbf{Adv}_c^{\text{nrnd}}(\mathcal{A}) = \Pr[\mathcal{A}^{\text{nReal}} \Rightarrow 1] - \Pr[\mathcal{A}^{\text{nRandom}} \Rightarrow 1]$.

In XCBv1 and XCBv2 , $c(S) = e(S) \parallel e(\text{incr}(S)) \parallel e(\text{incr}^2(S)) \parallel \dots$, where e is a block cipher and $\text{incr}(S) = S[0, n - 33] \parallel (S[n - 32, n - 1] + 1 \bmod 2^{32})$. It is easy to verify that c is ivRND secure but not nRND secure.

STRND. Let $\mathbf{E} : \mathcal{K} \times \mathcal{T} \times \mathcal{X} \rightarrow \mathcal{X}$ be a tweakable enciphering mode, and let $\mathbf{D}$ be its inverse. The advantage of distinguishing $\mathbf{E}$ from random bits is

$$\mathbf{Adv}_{\mathbf{E}}^{\text{strnd}}(\mathcal{A}) = \Pr_{K \xleftarrow{\$} \mathcal{K}} [\mathcal{A}^{\mathbf{E}_K(\cdot, \cdot), \mathbf{D}_K(\cdot, \cdot)} \Rightarrow 1] - \Pr[\mathcal{A}^{\$(\cdot, \cdot), \$(\cdot, \cdot)} \Rightarrow 1]$$

where $\$(T, P)$ returns a random string of length $|P|$. We insist that $\mathcal{A}$ makes no pointless queries, regardless of oracle responses, and $\mathcal{A}$ asks no query (T, P) outside of $\mathcal{T} \times \mathcal{X}$. We extend the definition above in the usual way to its resource-bounded versions. We have the following.

Lemma 4. *[STPRP-security $\approx$ STRND-security [18]] Let $\mathbf{E} : \mathcal{K} \times \mathcal{T} \times \mathcal{X} \rightarrow \mathcal{X}$ be a tweakable enciphering mode and let $q \geq 1$. Then for adversary $\mathcal{A}$ making q queries*

$$|\mathbf{Adv}_{\mathbf{E}}^{\text{stprp}}(\mathcal{A}) - \mathbf{Adv}_{\mathbf{E}}^{\text{strnd}}(\mathcal{A})| \leq \frac{q(q-1)}{2^{n+1}}$$

where n is the length of the shortest string in $\mathcal{X}$.

Proof of Lemma 3.

Proof. . The adversary $\mathcal{A}$ queries the encryption and decryption oracles of $\text{PIV}[\tilde{e}, \tilde{d}, c]$. In the following, we will replace the components in PIV with ideal components one-by-one, so that $\mathcal{A}$ interacts with two oracles in the subsequent games.

- Game G1: the encryption and decryption of $\text{PIV}[\tilde{e}, \tilde{d}, c]$.
- Game G2: the encryption and decryption of $\text{PIV}[\$, \tilde{d}, c]$.
- Game G3: the encryption and decryption of $\text{PIV}[\$, \$, c]$.
- Game G4: the encryption and decryption of $\text{PIV}[\$, \$, \$]$.
- Game G5: $\$$ and $\$$.

By regular reductions, there exist adversaries $\mathcal{B}$, $\mathcal{C}$ and $\mathcal{D}$, each making q queries, such that

$$\begin{aligned}
\Pr[\mathcal{A}^{G^1} \Rightarrow 1] - \Pr[\mathcal{A}^{G^2} \Rightarrow 1] &\leq \mathbf{Adv}_{\tilde{e}}^{\text{strnd}}(B), \\
\Pr[\mathcal{A}^{G^2} \Rightarrow 1] - \Pr[\mathcal{A}^{G^3} \Rightarrow 1] &\leq \mathbf{Adv}_{\tilde{d}}^{\text{strnd}}(C), \\
\Pr[\mathcal{A}^{G^3} \Rightarrow 1] - \Pr[\mathcal{A}^{G^4} \Rightarrow 1] &\leq \mathbf{Adv}_c^{\text{ivrnd}}(D).
\end{aligned}$$

G4 and G5 are identical, so we have

$$\mathbf{Adv}_{\text{PIV}[\tilde{e}, \tilde{d}, c]}^{\text{strnd}}(\mathcal{A}) \leq \mathbf{Adv}_{\tilde{e}}^{\text{strnd}}(B) + \mathbf{Adv}_{\tilde{d}}^{\text{strnd}}(C) + \mathbf{Adv}_c^{\text{ivrnd}}(\mathcal{D}).$$

By Lemma 4, we have

$$\mathbf{Adv}_{\text{PIV}[\tilde{e}, \tilde{d}, c]}^{\text{stprp}}(\mathcal{A}) \leq \mathbf{Adv}_{\tilde{e}}^{\text{stprp}}(B) + \mathbf{Adv}_{\tilde{d}}^{\text{stprp}}(C) + \mathbf{Adv}_c^{\text{ivrnd}}(\mathcal{D}) + \frac{3q^2}{2^{n+1}}.$$

□

References

1. Ashur, T., Dunkelman, O., Luykx, A.: Boosting authenticated encryption robustness with minimal modifications. In: Katz, J., Shacham, H. (eds.) CRYPTO 2017, Part III. LNCS, vol. 10403, pp. 3–33. Springer, Cham (2017). https://doi.org/10.1007/978-3-319-63697-9_1
2. Bao, Z., Guo, C., Guo, J., Song, L.: TNT: how to tweak a block cipher. In: Canteaut, A., Ishai, Y. (eds.) EUROCRYPT 2020. LNCS, vol. 12106, pp. 641–673. Springer, Cham (2020). https://doi.org/10.1007/978-3-030-45724-2_22
3. Bhati, A.S., Verbauwhede, M., Andreeva, E.: Breaking, repairing and enhancing XCBv2 into the tweakable enciphering mode GEM. Cryptology ePrint Archive, Paper 2024/1554 (2024). <https://eprint.iacr.org/2024/1554>
4. Bhaumik, R., Nandi, M.: An inverse-free single-keyed tweakable enciphering scheme. In: Iwata, T., Cheon, J.H. (eds.) ASIACRYPT 2015, Part II. LNCS, vol. 9453, pp. 159–180. Springer, Heidelberg (2015). https://doi.org/10.1007/978-3-662-48800-3_7
5. Chakraborty, D., Hernandez-Jimenez, V., Sarkar, P.: Another look at XCB. Cryptogr. Commun. **7**(4), 439–468 (2015). <https://doi.org/10.1007/S12095-015-0127-8>
6. Chakraborty, D., Sarkar, P.: A new mode of encryption providing a tweakable strong pseudo-random permutation. In: Robshaw, M. (ed.) FSE 2006. LNCS, vol. 4047, pp. 293–309. Springer, Heidelberg (2006). https://doi.org/10.1007/11799313_19
7. Chakraborty, D., Sarkar, P.: HCH: a new tweakable enciphering scheme using the hash-counter-hash approach. IEEE Trans. Inf. Theory **54**(4), 1683–1699 (2008). <https://doi.org/10.1109/TIT.2008.917623>
8. Chen, Y.L., et al.: Proposal of requirements for an Accordion mode: discussion draft for the NIST Accordion mode workshop 2024 (2024)
9. Crowley, P., Biggers, E.: Adiantum: length-preserving encryption for entry-level processors. IACR Trans. Symmetric Cryptol. **2018**(4), 39–61 (2018). <https://doi.org/10.13154/TOSC.V2018.I4.39-61>

10. Crowley, P., Huckleberry, N., Biggers, E.: Length-preserving encryption with HCTR2. *IACR Cryptol. ePrint Arch.* p. 1441 (2021). <https://eprint.iacr.org/2021/1441>
11. Datta, N., Dutta, A., Ghosh, S., Nandi, H.: Optimally secure TBC based accordion mode. *IACR Cryptol. ePrint Arch.* p. 2053 (2024). <https://eprint.iacr.org/2024/2053>
12. Dobraunig, C., Matusiewicz, K., Mennink, B., Tereschenko, A.: Efficient instances of docked double decker with AES. *IACR Cryptol. ePrint Arch.* p. 84 (2024). <https://eprint.iacr.org/2024/084>
13. Fleischmann, E., Forler, C., Lucks, S., Wenzel, J.: McOE: a family of almost fool-proof on-line authenticated encryption schemes. *Cryptology ePrint Archive*, Paper 2011/644 (2011). <https://eprint.iacr.org/2011/644>
14. Gueron, S., Langley, A., Lindell, Y.: AES-GCM-SIV: specification and analysis. *IACR Cryptol. ePrint Arch.* p. 168 (2017). <http://eprint.iacr.org/2017/168>
15. Gunsing, A., Daemen, J., Mennink, B.: Deck-based wide block cipher modes and an exposition of the blinded keyed hashing model. *IACR Trans. Symmetric Cryptol.* **2019**(4), 1–22 (2019). <https://doi.org/10.13154/TOSC.V2019.I4.1-22>
16. Halevi, S.: EME*: extending EME to handle arbitrary-length messages with associated data. In: Canteaut, A., Viswanathan, K. (eds.) *INDOCRYPT 2004*. LNCS, vol. 3348, pp. 315–327. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30556-9_25
17. Halevi, S.: Invertible universal hashing and the TET encryption mode. In: Menezes, A. (ed.) *CRYPTO 2007*. LNCS, vol. 4622, pp. 412–429. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-74143-5_23
18. Halevi, S., Rogaway, P.: A tweakable enciphering mode. In: Boneh, D. (ed.) *CRYPTO 2003*. LNCS, vol. 2729, pp. 482–499. Springer, Heidelberg (2003). https://doi.org/10.1007/978-3-540-45146-4_28
19. Halevi, S., Rogaway, P.: A parallelizable enciphering mode. In: Okamoto, T. (ed.) *CT-RSA 2004*. LNCS, vol. 2964, pp. 292–304. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24660-2_23
20. IEEE 1619.2: IEEE standard for wide-block encryption for shared storage media (2011)
21. Jha, A., Khairallah, M., Nandi, M., Saha, A.: Tight security of TNT and beyond - attacks, proofs and possibilities for the cascaded LRW paradigm. In: Joye, M., Leander, G. (eds.) *EUROCRYPT 2024, Part I*. LNCS, vol. 14651, pp. 249–279. Springer, Cham (2024). https://doi.org/10.1007/978-3-031-58716-0_9
22. Landecker, W., Shrimpton, T., Terashima, R.S.: Tweakable blockciphers with beyond birthday-bound security. In: Safavi-Naini, R., Canetti, R. (eds.) *CRYPTO 2012*. LNCS, vol. 7417, pp. 14–30. Springer, Heidelberg (2012). https://doi.org/10.1007/978-3-642-32009-5_2
23. Lee, B.: A BBB secure accordion mode from hctr. *NIST Workshop on the Requirements for an Accordion Cipher Mode* (2024)
24. Liskov, M., Rivest, R.L., Wagner, D.: Tweakable block ciphers. In: Yung, M. (ed.) *CRYPTO 2002*. LNCS, vol. 2442, pp. 31–46. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-45708-9_3
25. McGrew, D.A., Fluhrer, S.R.: The extended codebook (XCB) mode of operation. *IACR Cryptol. ePrint Arch.* p. 278 (2004). <http://eprint.iacr.org/2004/278>
26. McGrew, D.A., Fluhrer, S.R.: The security of the extended codebook (XCB) mode of operation. In: Adams, C., Miri, A., Wiener, M. (eds.) *SAC 2007*. LNCS, vol. 4876, pp. 311–327. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-77360-3_20

27. McGrew, D.A., Viega, J.: The security and performance of the Galois/counter mode (GCM) of operation. In: Canteaut, A., Viswanathan, K. (eds.) *INDOCRYPT 2004*. LNCS, vol. 3348, pp. 343–355. Springer, Heidelberg (2004). https://doi.org/10.1007/978-3-540-30556-9_27
28. Minematsu, K., Matsushima, T.: Tweakable enciphering schemes from hash-sum-expansion. In: Srinathan, K., Rangan, C.P., Yung, M. (eds.) *INDOCRYPT 2007*. LNCS, vol. 4859, pp. 252–267. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-77026-8_19
29. Namprempre, C., Rogaway, P., Shrimpton, T.: Reconsidering generic composition. In: Nguyen, P.Q., Oswald, E. (eds.) *EUROCRYPT 2014*. LNCS, vol. 8441, pp. 257–274. Springer, Heidelberg (2014). https://doi.org/10.1007/978-3-642-55220-5_15
30. Nandi, M.: An efficient SPRP-secure construction based on pseudo random involution. *IACR Cryptol. ePrint Arch.* p. 92 (2008). <http://eprint.iacr.org/2008/092>
31. Nir, Y., Langley, A.: Chacha20 and poly1305 for IETF protocols. RFC **7539**, 1–45 (2015). <https://doi.org/10.17487/RFC7539>
32. Pietrzak, K., Sjödin, J.: Range extension for weak PRFs; the good, the bad, and the ugly. In: Naor, M. (ed.) *EUROCRYPT 2007*. LNCS, vol. 4515, pp. 517–533. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-72540-4_30
33. Sarkar, P.: Improving upon the TET mode of operation. In: Nam, K.-H., Rhee, G. (eds.) *ICISC 2007*. LNCS, vol. 4817, pp. 180–192. Springer, Heidelberg (2007). https://doi.org/10.1007/978-3-540-76788-6_15
34. Shrimpton, T., Terashima, R.S.: A modular framework for building variable-input-length tweakable ciphers. In: Sako, K., Sarkar, P. (eds.) *ASIACRYPT 2013*, Part I. LNCS, vol. 8269, pp. 405–423. Springer, Heidelberg (2013). https://doi.org/10.1007/978-3-642-42033-7_21
35. Wang, P., Feng, D., Wu, W.: HCTR: a variable-input-length enciphering mode. In: Feng, D., Lin, D., Yung, M. (eds.) *CISC 2005*. LNCS, vol. 3822, pp. 175–188. Springer, Heidelberg (2005). https://doi.org/10.1007/11599548_15